Longest Peak

AlgoExpert · Medium · Arrays

The problem

- Length of the longest strictly-increasing-then-strictly-decreasing run (needs >= 3 items).

Intuition

- A peak is a strict rise then a strict fall, so find each tip and expand both ways.

Brute force vs optimal

- Brute force: from every index, expand a peak → $O(n^2)$
- Optimal: only expand from tips, skip past each peak → $O(n)$
- Each element is visited at most twice

Approach

- 1. Find indices higher than both neighbours
- 2. Expand left while values keep decreasing outward
- 3. Expand right likewise
- 4. Track the longest span

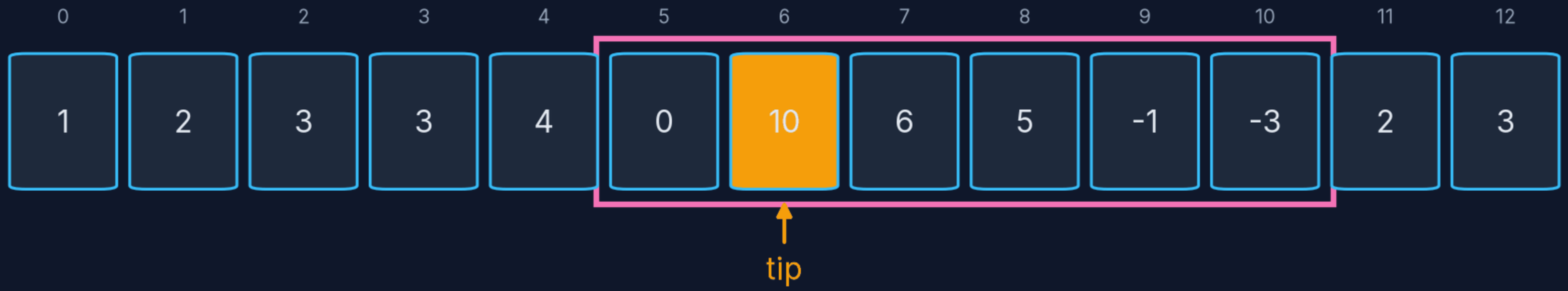
Worked example




```
peak 3..5, length 3
best = 3
```

Tip 4 at 4 → peak spans 3...5 (length 3)

Worked example




```
peak 5..10, length 6
best = 6
```

Tip 10 at 6 → peak spans 5…10 (length 6)

Java solution

```java
public class LongestPeak {
    public static int longestPeak(int[] a) {
        int best = 0, i = 1;
        while (i < a.length - 1) {
            if (a[i - 1] < a[i] && a[i] > a[i + 1]) {
                int l = i - 1, r = i + 1;
                while (l > 0 && a[l - 1] < a[l]) l--;
                while (r < a.length - 1 && a[r + 1] < a[r]) r++;
                best = Math.max(best, r - l + 1);
                i = r;
            } else i++;
        }
        return best;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Strict inequalities: plateaus (3, 3) aren't peaks
- Need ≥ 3 elements; only rising or only falling → 0
- Jump i to r to avoid rescanning

Interview tips & follow-ups

- Draw the array as a mountain range while explaining
- Follow-up: longest valley; count peaks

Recap

- A peak is a strict rise then a strict fall, so find each tip and expand both ways.
- Complexity: O(n)